Advanced Problem-Solving Lab – High-IQ Track

Activity H1 – Solve the "Hidden Problem" Behind the Stated Problem

Title: From Surface Problem to Real Problem

Goal

Train yourself to see the *real* problem, not just the stated one.

Instructions

You are given a "client statement":

"Our exam system is too slow. We want a Python script to speed it up."

1. Identify at least 3 possible *real* problems this statement might hide.
   - Hidden problem 1: Database queries are slow
   - Hidden problem 2: Too many users at same time
   - Hidden problem 3: Inefficient code
2. For each hidden problem, state:
   - What data would you need to confirm it?
     - Problem 1: Query execution time
     - Problem 2: Number of active users
     - Problem 3: Code execution time
3. Choose ONE hidden problem and re-frame it as a clear technical problem statement (2–3 lines):

New problem statement:

The system is slow due to inefficient database queries which increases response time. The goal is to optimal query performance

4. Design a very small Python experiment that could validate or reject your reframed problem (pseudo-code is fine):

python

 # Experiment idea (what will you measure, on what data, how?)

START

Measure current query time

Optimize query

Measure new time

Compare results

END

Activity H2 – Multiple Competing Solutions

Title: Design 3 Solutions Before Coding 1

Goal

Avoid fixation. Explore at least three distinct solution strategies.

Instructions

Problem:

"You receive a stream of log lines (text). You must detect 'suspicious' activity in real time."

1. Give three different solution ideas, each using a distinct main technique or data structure.
   - Approach A (e.g., rule-based / regex / thresholds):
     Rule - based detection
   - Approach B (e.g., statistical / sliding window / counters):
     Sliding Window method
   - Approach C (e.g., simple ML / anomaly detection concept):
     Simple ML model
2. For each approach, list pros and cons (at least one each).
   - Approach A – Pros: Simple Cons: limited
   - Approach B – Pros: Real time Cons: Complex
   - Approach C – Pros: Accurate Cons: Needs data
3. Choose one approach to implement first and justify:

I choose approach A *because*

it is simple and easy to implement.

4. Sketch high-level Python structure (modules/classes, not full code):

python

 # Modules / classes / key functions list

```
Python
read_logs()
detect_suspicious()
print_result()
```

Activity H3 – Self-Constrained Challenge: "Elegant Code Only"

Title: Solve with Elegance (Clarity, Minimalism, Correctness)

Goal

Produce code that is not only correct, but also minimal and beautiful.

Problem

Implement a mini "pipeline" for processing a list of tasks:

- Each task has an ID, description, and status ("todo", "doing", "done").
- You must:
  - Load tasks from a JSON file.
  - Filter by status.
  - Sort by ID.
  - Print a clean summary.

Constraints (you impose on yourself)

1. Maximum 25 lines of *real* Python (excluding blanks and comments).
2. No function longer than 7 lines.
3. Names must be meaningful and short.

Steps

1. Design your data model in 2–3 lines:

   Tasks stored as dictionary inside list

2. Decide at least one thing you will *not* do to keep it minimal:

   Avoid complex classes

3. Write your code here (respecting constraints):

python

 # Elegant pipeline solution

4. Reflection:
   - What did you have to sacrifice (if anything) to keep elegance?

   Code is short, simple and readable

3) Python

```
import json
tasks = json.load(open("tasks.json"))
tasks = [t for t in tasks if t["status"] == "todo"]
tasks = sorted(tasks, key=lambda x: x["id"])
for t in tasks:
    print(t["id"], t["description"], t["status"])
```

Activity H4 – Complexity and Scaling Thought Experiment

Title: How Will Your Solution Break at Scale?

Goal
Develop realistic intuition about performance and scaling.

Instructions
Take ANY non-trivial function or algorithm you have recently written (search, summarization, scoring, etc.).

1. Describe what it does, in *one line*:
   Find sum of numbers
2. Estimate its time complexity in Big-O notation, and justify in 1–2 lines:
   - Big-O: O(n)
   - Because: loop runs n times
3. Scaling scenarios – imagine input sizes:
   - n = 10^3 (1,000)
   - n = 10^6 (1,000,000)
   - n = 10^9 (1,000,000,000)

For each, predict: will it still be usable (Y/N) and why?

n	Usable (Y/N)	Reason (1 short sentence)
10^3	Yes	fast
10^6	Yes	Moderate time
10^9	No	Too slow

4. Suggest one change to improve scalability and briefly describe how it helps:
   Use optimized methods like parallel processing

Activity H5 – Adversarial Thinking: Break Your Own Design

Title: Attack Your Own Program

Goal

Develop robustness and defensive mindset.

Instructions

Choose one of your own projects or mini-tools (file reader, API client, CLI, etc.).

1. Write a short description of what it is supposed to do:

   Appointment system

2. Now wear the hat of an *attacker* or malicious user.
   List 5 ways to misuse / break it:

   - Attack 1: Empty input
   - Attack 2: Invalid data
   - Attack 3: Large input
   - Attack 4: Wrong file path
   - Attack 5: Duplicate data

3. For each attack, suggest one defence in plain language:

   - Defence for attack 1: Input validation
   - Defence for attack 2: Error messages
   - Defence for attack 3: Limit input size
   - Defence for attack 4: Exception handling
   - Defence for attack 5: Check duplicate

4. Implement at least one of these defences in your code and paste the updated snippet:

python

 # Hardened code snippet:

```
name = input ("Enter name : ")
if name == " ":
    print (" Invalid input ")
```

Activity H6 – Cross-Domain Transfer

Title: Apply Known Pattern to a New Domain

Goal

Practice lifting a known idea/pattern from one domain to a different one.

Instructions

Pick a pattern you already know well, e.g.:

- Sliding window
- Two-pointer technique
- Dynamic programming table
- Producer–consumer
- Map–reduce style
- Pipeline of filters

1. Name of pattern: Sliding window
2. Original domain where you learned it (e.g., arrays, strings):

   Arrays

3. New domain to apply it in (e.g., logs, time series, learning activities, medical data):

   log data

4. Describe how you would adapt the pattern to this new domain, in 5–8 lines (can include pseudo-code):

   we take a fixed window of logs and check supicious activity in that window continuously

5. Optional: implement a small prototype showing this transfer:

python

 # Pattern transfer prototype

```
python =
logs = [1, 2, 3, 4, 5]
for i in range (len (logs) - 2) :
    window = logs [i : i+3]
    print (window)
```

2. Write th

Activity H7 – Meta-Cognition: Your Personal Problem-Solving Manual

Title: Write Your Own "How I Solve Problems" Guide

Goal

Turn your tacit skills into an explicit, reusable system.

Instructions

1. List the first three things you usually do when you get a new problem (be honest):
   - Step 1: Read problem
   - Step 2: Think solution
   - Step 3: Start coding
2. Now, design your ideal 6–8 step process (what a "future you" would always do):

1. Understand problem
2. Identify inputs and outputs
3. Break into steps
4. choose approach
5. Write algorithm
6. Code
7. (optional) Test
8. (optional) _____

3. For each step, write one question you will ask yourself:
   - Step 1 question: what is the problem ?
   - Step 2 question: what are inputs /outputs ?
   - Step 3 question: what steps are needed ?
   - Step 4 question: which method to use ?
   - Step 5 question: Is algorithm correct ?
   - Step 6 question: Does code work ?
4. Final instruction to yourself (one sentence):

"When I feel stuck, I will I will break the problem into smaller parts and solve step by step